11 4 213

schedule_feature_research.sh — §11.4.213 FEATURE research-scheduling engine

Field	Value
Revision	2
Created	2026-07-16
Last modified	2026-07-16T12:00:00Z
Status	active
Script	`constitution/scripts/feature/schedule_feature_research.sh`
Authority	`constitution/Constitution.md` §11.4.213

Overview

`schedule_feature_research.sh` turns a plain-language FEATURE DESCRIPTION — delivered by the §11.4.140 registered action `FEATURE`, or by any other caller — into a **SCHEDULED** (never synchronously executed) deep, enterprise-grade research + implementation-planning effort.

```
feature description
   ├─(1) CREATE → a Type=Task workable item, by DELEGATING to the SAME
   │              §11.4.202 report_item.sh engine (never a duplicate
   │              implementation) — its description embeds the full
   │              research-and-planning WORK PROGRAM as acceptance
criteria
   ├─(2) SYNC + PUSH → delegated to report_item.sh exactly as §11.4.202
   │              performs them (absent credentials/client ⇒ honest SKIP,
   │              NEVER a faked push)
   └─(3) QUEUE  → append a row to the durable docs/requests/
feature_queue.md
                  (mirrors the §11.4.140 BACKGROUND durable-queue pattern)
```

The multi-day research / planning / documentation work described INSIDE the item is **not performed by this engine**. It is performed LATER by the project's standing autonomous loop (§11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.126) when it claims the item, driven to a genuinely COMPLETED-and-wired or explicitly evidence-backed CLOSED terminal state under the §11.4.197 research-completion mandate — a FEATURE item is NEVER left sitting un-wired in the backlog.

Prerequisites

- `python3` (+PyYAML) — reads the consumer config and parses the delegate's JSON result.
- The sibling `constitution/scripts/reporting/report_item.sh` engine (§11.4.202) — this engine DELEGATES to it rather than reimplementing item-creation / DB-sync / tracker-push.
- A **consumer-owned config** (the engine refuses to guess — §11.4.6):

```
mkdir -p .helix
cp constitution/scripts/feature/feature.example.yaml .helix/feature.yaml
$EDITOR .helix/feature.yaml     # db, id_prefix, research_doc_root, sync_command, trackers
```

Search order: `$HELIX_FEATURE_CONFIG` → `.helix/feature.yaml` → `config/feature/feature.yaml`. The SAME file is schema-compatible with `reporting.example.yaml` (§11.4.202) plus one addition, `research_doc_root` — it is passed straight through to `report_item.sh --config`.

Usage examples

```
# Schedule a feature research effort (Type=Task, Status=Queued):
bash constitution/scripts/feature/schedule_feature_research.sh \
  --description 'Add per-user configurable notification digest frequency (daily/weekly/off).'

# Preview without writing anything (no delegate invocation, no queue append):
bash constitution/scripts/feature/schedule_feature_research.sh \
  --description '...' --dry-run

# Explicit title + machine-readable result:
bash constitution/scripts/feature/schedule_feature_research.sh \
  --title 'Dark-mode theme toggle' \
  --description 'Add a dark-mode theme toggle to the settings screen.' --json
```

Flag	Meaning
`--description` / `--description-file` / stdin	the feature description (in that precedence)
`--title`	optional; derived from the description's first sentence when omitted
`--severity`, `--reported-by`	optional; passed through to the delegate
`--config`, `--db`	optional overrides (else the search order above)
`--no-sync`, `--no-tracker`	skip step 2's sync / tracker-push (passed through to the delegate)
`--dry-run`, `--json`	preview / machine-readable result

Edge cases (all honest, none faked)

Situation	Behaviour
No config found	**Exit 2** with the template path. The engine never guesses your DB path / research-doc root (§11.4.6).
The delegate (`report_item.sh`) is missing	**Exit 3** — FEATURE never falls back to reimplementing item-creation itself.
Item creation fails	**Exit 4**, propagated from the delegate; the FEATURE item was NOT created and NOT queued.
Sync command fails	**Exit 5**, propagated from the delegate; the item **IS** created and **IS** queued (the DB is the SSoT) — re-run the sync.
Tracker credentials unset	Verdict `SKIP` , reason `credentials_absent: unset env: NAME …` (via the delegate) — **variable names only, never values** (§11.4.10).
Tracker client not built	Verdict `SKIP` , reason `tracker_client_absent` (via the delegate).

A tracker `PASS` is recorded **only** after a real push command exits 0 — inherited from the delegate, never reimplemented here.

Internal behaviour

1. **Config** → this engine reads only its OWN key (`research_doc_root` , default `docs/research`) + `evidence_dir` ; every other key (`db` / `id_prefix` / `sync_command` / `trackers` / …) is read independently by the DELEGATED `report_item.sh` from the SAME file.
2. **Title + destination slug** → derived from the description's first sentence (mirrors `report_item.sh`); the destination is `<research_doc_root>/<slug>_<UTC-timestamp>_<PID>/` — the trailing `$$` (this process's PID) is a uniquifier: two identical-title requests scheduled within the same second would otherwise

collide on the same `(slug, seconds-resolution timestamp)` key and record the SAME research-doc destination on two distinct items; the PID is already resolvable at zero cost and keeps the path human-readable.

3. **Work program** → the full §11.4.213 clause-(2) mandate ((a)–(k)) is rendered verbatim and embedded into the report text passed to the delegate, together with the destination path and the scheduling-not-synchronous execution model.
4. **Delegate** → `report_item.sh --kind task --title … --report … --scope … --acceptance … --config … --json` — the SAME engine §11.4.202 uses, so the already-audited no-faked-push / decoupling / anti-bluff invariants are inherited, never duplicated.
5. **Queue** → append one row to `docs/requests/feature_queue.md` (created with a header + §11.4.44 revision line on first use). The title is the only user-influenced table cell; a literal `|` in it is escaped (`\|`) before the append so the row stays a well-formed 5-cell markdown row (an unescaped pipe would otherwise shift Destination / Scheduled / Status one column right) — the ITEM_ID / destination / timestamp / literal `Queued` cells are generated, never free-text, and need no escaping.
6. **Evidence** → `<evidence_dir>/<id>_<ts>/result.json` (this engine's own summary, embedding the delegate's own `report_item_result.json` verbatim) (§11.4.5 / §11.4.69).

Decoupling (§ . . / § . .)

The engine carries **zero project literals** and is inherited **by reference** (never copied). Every project-specific value is consumer-owned DATA. It also carries **zero duplicate item-creation / tracker-push logic** — that machinery lives exactly once, in `report_item.sh`. The `CM-FEATURE-DIRECTIVE` gate enforces both invariants (a hardcoded project literal, or a reimplemented tracker-push path, → gate FAILs).

Related

Path	Purpose
`constitution/actions/registry.yaml`	the `FEATURE` row + the shared §11.4.140 grammar
`constitution/scripts/action_prefix_lib.sh`	the §11.4.140 parser (all six forms)
`constitution/scripts/feature/feature.example.yaml`	the consumer config template
`constitution/scripts/gates/cm_feature_directive.sh`	the pre-build gate (+ its §1.1 mutation test)
`constitution/scripts/reporting/report_item.sh`	the DELEGATED §11.4.202 item-creation + full-sync engine
`constitution/scripts/workable-items/`	the DB SSoT binary (§11.4.93)

Last verified: 2026-07-16 — end-to-end against a real temp SQLite DB (via `sqlite3 <tmp>.db < schema.sql`): a dry-run preview, three real items created (`FTX-001` / `FTX-002` / `FTX-003`) with the correct Type=Task / Status=Queued, a configured tracker SKIPped honestly with `credentials_absent`, special characters (quotes / newlines / backslash) in the description round-tripped correctly through the DB + the JSON evidence, and `docs/requests/feature_queue.md` accumulated one row per scheduled item — all throwaway artefacts removed after the run.